

First, reproduce learning in a simpler setting.

A recursive language model (RLM) can use code and ask helper models to solve parts of a problem. We eventually want to train those choices.

Today's question

Can we use a published training recipe to improve a model's math answers on questions excluded from training?

We use the authors' code for **Dr. GRPO**, a reinforcement learning (RL) method. It learns from scores assigned to the model's own answers.

This is a small reproduction of an existing method. It establishes a reference for future work on recursive models.

Liu et al., *Understanding R1-Zero-Like Training: A Critical Perspective* (2025).

Official code; our run is much smaller than the paper's experiments.

Reward correct attempts, then change the model.

Illustrative question: Solve $3x + 2 = 14$. The correct answer is $x = 4$.

The model's attempts	Checker score	Learning direction
Two answers end in $x = 4$.	1 each	Make these more likely.
Two end in $x = 5$ or $x = 6$.	0 each	Make these less likely.

Each attempt is compared with the average score for that question. These differences guide changes to the model's numerical settings, or **weights**.

The checker judges the final answer. We do not supply worked solutions for the model to copy.

Simplified illustration with four attempts; our runs use eight. If every attempt receives the same score, this question supplies no relative learning signal.

We train on math, then test on different questions.

We trained **Qwen2.5-Math-1.5B**, a relatively small pretrained math model, on one 40 GB A100 GPU.

Training: 512 questions, 32 weight updates.

The questions came from the authors' 12,000-question math collection. The model tried eight answers to each question.

Testing: 128 questions from MATH500.

These questions were excluded from training and progress checks. We tested the starting model and the final trained model on exactly the same questions.

The prompt, answer checker, and response-length limit stayed the same. We cannot rule out exposure to these questions during the model's earlier pretraining.

Training improved answers on the separate test.

Correct answers out of 128 MATH500 questions.

Before training

41 / 128

After training

80 / 128

44 answers became correct; 5 became incorrect.

The score counts answers accepted by the authors' final-answer checker. A correct answer does not certify every step of the explanation.

This is a clear improvement in this test, from one short training run. It needs an independent repeat.

Final checkpoint after the planned 32 updates, not the best-scoring checkpoint. A 500-question comparison is now running.

A higher score does not always mean a new skill.

An actual test question: Write $3/20$ as a decimal.

The saved responses, summarized

Before: The model repeated the question until it hit the length limit.

After: It explained the division and answered 0.15.

For 36 of the 44 newly correct answers, the original response had hit the length limit. The gain is useful, but it need not mean newly learned math.

Next: Test a simpler prompt and repeat training. For RLMS, ask whether rewards teach useful subquestions, not just more usable answers.

This reproduces a training recipe at small scale, not the paper's full results or learning to decompose problems.